

ECE 202 Project Proposal: Simple Electrical Impedance Tomography

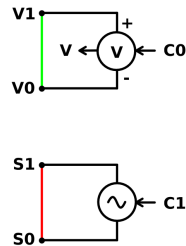
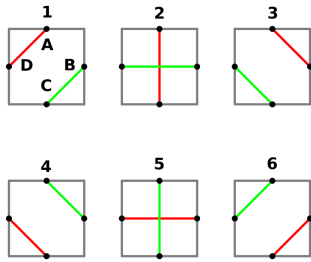
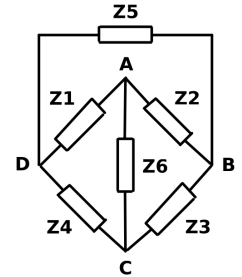
Team Members: Jonah Spector, Connor Cassidy, Christian Rayner

EIR Mentor: Chuck Duey (Approved Project)

Electrical impedance tomography is a method for medical imaging which visualizes the internal resistivity of the human body, namely for real-time monitoring of the lungs. It measures the voltage on n -electrodes and sends an alternating current through 2 other electrodes. This results in a non-linear, ill-posed boundary value problem which, when solved, gives a 2D tomogram of resistivities. We have decided to make a 4 electrode electrical impedance tomography device for our project using a lumped element model to calculate the location of an object. Our visualization will, at the very least, show four pixels, each representing the four quadrants outlined on the sides by the electrodes. Each pixel will show if it is likely that there is an object of higher resistivity (OHR) contained in that quadrant. This visualization, we hope, will update at a reasonable framerate of 5-60 fps. If we succeed at this, we may expand our project to have more electrodes, a better framerate, a complete inverse algorithm, or improve it in other ways.

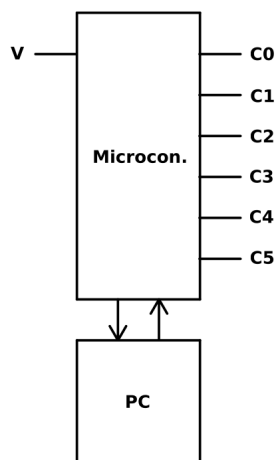
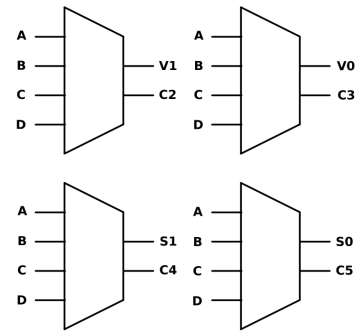
Simple Electrical Impedance Tomography

In order to calculate the location of the OHR, we plan to use a lumped element model for the actual medium of known concentration saline. By sending an AC signal at a frequency high enough to avoid electrolysis of the saline, and low enough to be captured by the digital multimeter ($\sim 5\text{-}10\text{ kHz}$), we can determine which of Z_{1-4} is the highest, or even possibly reconstruct every impedance in the lumped element model.



We plan to send the signal across 2 electrodes, and measure the voltage across the remaining 2 electrodes. With 4 electrodes, this gives us 6 configurations (or 6 unique voltage measurements). We will use four, 1-to-4 DEMUXs to switch between the 6 configurations.

A microcontroller will control the 4 synchronous DEMUXs at 30-360 Hz (configurations per second). Each time the configuration switches, the microcontroller records a new voltmeter measurement and saves the wave. Each frame, the waves from all 6 configurations are bundled, by the microcontroller, into a packet sent to the computer. Then, the computer takes each packet and computes a visualization (5-60 fps).



The data will be processed into a 4 pixel image showing which resistivity (Z_{1-4}) is the highest (where the OHR is). Alternatively, and if possible, the actual values of all 6 resistivities will be computed, given the varying cross-sectional areas of the saline in the test rig and the distance between the electrodes. The OHR will be made of water-proof plastic, and a 3D-printed, waterproof test rig will be constructed to hold the electrodes and saline.

The test rig will consist of a container measuring 16 x 16 x 8 cm, with 2 cm fillets on all internal edges. It will be filled with 3 cm of saline. Electrodes will be made of graphite to prevent corrosion, be positioned on all four sides, and have a 2 mm diameter.

Material List: 4 x 1-to-4 DEMUX, 4 x Graphite Electrodes, Electrode Sheathing, AC Power Supply, Digital Multimeter, Microcontroller, (Multiple) 3D Printed Test Rig (Container and OHR), Known Concentration Saline, Computer.

Goals	
Must	Test rig, 4 electrode network, independent AC power supply, digital voltmeter, DEMUX to switch configurations (30 Hz), Python visualizer (4 pixel display), microcontroller system integration.
Want	Higher speed configuration switching (~360 Hz), faster Python/C visualizer, custom AC power supply, full lumped element model solver.
Would be nice	More electrodes (5), bigger test rig, full BVP solver.
Miracle	Even more electrodes, more DEMUXs, organic tests (arteries, veins, trachea). Full EIT with lung test (impossible).

Deadlines	
Week 2 (2/8 to 2/14)	Advanced proof of concept, test rig, device requirements.
Week 3-4	Hard-/software requirements, order DEMUXs, pseudo-code manual algorithm, AC power supply, digital multimeter.
Week 5	Manual MUX, manual algorithm (PyVISA).
Week 6 (Mid-report)	Implement microcontroller, microcontroller program, computer real-time visualization algorithm.
Week 7 (Spring Break)	Catch-up week.
Week 8	Integrate shipped DEMUXs, continue programming.
Week 9	Complete system integration, debugging (device complete).
Week 10	Improve speed of microcontroller, DEMUX, visualizer.
Week 11	Finalize demo presentation and project report.
Week 12 (Demo)	Catch-up week.